

ThesisFlow — Análise de Segurança de Software

Grupo 06 — UNIPAMPA

- Marcus Vinicius Morini Querol Junior
- Bernardo Gomes Dorneles
- Gustavo Fernandes dos Anjos
- Fade Hassan Husein Kanaan
- Rodrigo Thoma da Silva
- Artur Wahlbrink Kraemer

Escopo do Trabalho

Análise abrangente de segurança cibernética do sistema ThesisFlow, cobrindo da modelagem de ameaças e priorização de riscos até o código seguro, DAST e esteira DevSecOps.

STRIDE

NIST CSF 2.0

RBAC

OWASP ZAP

DEVSECOPS

ETAPA 1 — CONTEXTO DO SISTEMA

O Sistema Escolhido: ThesisFlow

Estudante

Registra plano de trabalho acadêmico (24 meses), submete comprovantes de atividades creditáveis e acompanha prazos de qualificação.

Orientador

Valida atividades creditáveis, avalia relatórios de progresso do orientando e vincula produções científicas desenvolvidas.

Coordenador

Administra o programa de pós-graduação, aprova prorrogações de prazo, configura políticas de créditos e emite relatórios institucionais.

Justificativa da Escolha

O ThesisFlow é um sistema real desenvolvido pelo próprio grupo. Sua escolha permitiu auditoria profunda de arquitetura, gestão de dados pessoais sensíveis (LGPD) e aplicação de controles de acesso rígidos em múltiplos perfis.

Principais Ameaças & Casos de Abuso (STRIDE)

Caso de Abuso 01 — IDOR

INFORMATION
DISCLOSURE

Estudante malicioso altera parâmetros na API (ex: `/students/student_456/profile`) para visualizar dados acadêmicos e pessoais confidenciais de outros estudantes.

Caso de Abuso 02 — Forja de Upload

TAMPERING

Submissão de arquivos maliciosos ou executáveis renomeados como PDF para burlar a aprovação de comprovantes e obter créditos indevidos de atividades.

Mapeamento de Ameaças com STRIDE

Catologação detalhada de 12 ameaças abrangendo Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service e Elevation of Privilege ao longo de toda a superfície da API.

Riscos Prioritários & NIST CSF 2.0

Riscos de Severidade Crítica Priorizados

- **R07** Acesso indevido a dados de outro estudante via falha de IDOR.
- **R08** Alteração fraudulenta de status acadêmico sem autorização.
- **R03** Upload de comprovantes com arquivos executáveis/maliciosos.

Alinhamento com as Funções NIST CSF 2.0

- **Protect (PR.AA / PR.DS):** Controle de Acesso Rígido no backend e validação estrita de entradas.
- **Detect (DE.CM):** Logs de auditoria estruturados para tentativas de violação.
- **Respond (RS.MA):** Procedimentos de contenção e revogação imediata de acesso.

ETAPA 3 — ARQUITETURA SEGURA

Decisões de Arquitetura Segura

Defesa em Profundidade

Múltiplas camadas independentes de validação: Autenticação centralizada via Firebase Auth (Tokens JWT) + Autorização RBAC estritamente no backend.

Validação Suprema no Servidor

Interface frontend tratada como ambiente não confiável. O backend inspeciona o token JWT, verifica a regra de papel (Role-Based Access Control) e valida a propriedade do recurso antes de qualquer processamento.

Fluxo da Requisição Autenticada

Cliente Web —(HTTPS/JWT)—> API Gateway —(Verify Token)—> Middleware RBAC (@exige_perfil) —> Controller

Práticas de Código Seguro & Suíte Pytest

Práticas de Implementação (OWASP / CWE)

- **Decorador RBAC:**
@exige_perfil(["COORDINATOR"])
impedindo desvio de privilégios.
- **Upload Seguro (CWE-434):** Checagem de extensões permitidas + Inspeção de Magic Bytes (assinatura binária do arquivo).
- **Sanitização:** Bloqueio de Path Traversal em nomes de comprovantes.

Testes Automatizados de Segurança

```
def test_idor_attempt_returns_403():  
    response = client.get("/students/456", headers=auth_student_123)  
    assert response.status_code == 403  
    assert "UNAUTHORIZED_IDOR" in audit_logs
```

100% SUCESSO NOS TESTES DE SEGURANÇA

Resultados da Verificação (OWASP ZAP)

Achados da Varredura Dinâmica

- **ALERTA CORS** Configuração de CORS permissiva no ambiente de testes inicial.
- **ALERTA CSP** Ausência de cabeçalhos rígidos de Content Security Policy.
- **HEADERS HTTP** Ausência dos cabeçalhos X-Content-Type-Options: nosniff e HSTS.

Correções & Triagem Crítica

- **Restrição de CORS:** Configuração estrita de origens permitidas vinculadas ao domínio oficial do frontend.
- **Cabeçalhos Rígidos:** Injeção dos headers de segurança OWASP recomendados.
- **Triagem de Falsos Positivos:** Validação manual para desconsiderar alertas irrelevantes.

Regras de Detecção de Intrusão

Regra D01 — Abuso de IDOR

Alerta acionado quando um usuário acumula **> 3 falhas de autorização em 5 minutos** ao solicitar dados de terceiros.

AÇÃO: REVOGAÇÃO TEMPORÁRIA DE SESSÃO

Regra D02 — Anomalia de Tráfego

Alerta acionado quando o volume de requisições por IP ultrapassa a taxa de limitação (*rate limiting*) estabelecida.

AÇÃO: BLOQUEIO HTTP 429 (TOO MANY REQUESTS)

Trilha de Auditoria Estruturada

Registros padronizados em JSON gravados em servidor auditável contendo `timestamp`, `uid`, `ip`, `endpoint`, `action` e `status_code`.

Pipeline DevSecOps & Quality Gates

1. Secret Scanning

GITLEAKS

Bloqueia a aprovação de Pull Requests caso detecte tokens ou chaves secretas no código.

2. SAST & Unit Tests

BANDIT + PYTEST

Analisa o código estático e executa a suíte de testes de autorização e upload seguro.

3. DAST Baseline Scan

OWASP ZAP

Roda varredura dinâmica contra a API implantada em ambiente de testes temporário.

Quality Gates de Bloqueio Automático

O código só avança para homologação ou produção se todos os 4 portões de qualidade forem aprovados com 100% de sucesso.

Evolução do ThesisFlow na Disciplina

Início da Disciplina (MVP Vulnerável)

- Ausência de validação de propriedade de recursos (IDOR).
- Uploads sem inspeção binária de arquivos.
- Falta de auditoria padronizada e testes automatizados.

Estado Final (Sistema Seguro DevSecOps)

- Modelagem STRIDE e plano NIST CSF 2.0 consolidados.
- Decoradores RBAC no servidor e validação de magic bytes.
- Logs de auditoria e regras de detecção ativas.
- Esteira CI/CD no GitHub Actions com 4 Quality Gates.

O Que o Grupo Aprendeu

Shift-Left Security

Antecipar a segurança para a fase de modelagem reduz custos de refatoração e blinda o software antes de escrever a primeira linha de código.

Segurança por Design

Projetar arquitetura em camadas (RBAC + Firebase Auth) garante que o sistema permaneça protegido independente da interface.

DevSecOps Automatizado

Integrar verificações estáticas, dinâmicas e testes de invasão na esteira CI/CD assegura conformidade sem prejudicar a agilidade.

Monitoramento & Resposta

Prevenir ataques é essencial, mas possuir trilhas de auditoria para detectar anomalias é vital para a resiliência operacional.

ETAPA 7 — ENCERRAMENTO

Conclusão & Repositório do Projeto

Conclusão do Trabalho

O ThesisFlow atingiu um estado maduro e seguro, atendendo rigorosamente a todos os requisitos e entregáveis estipulados na disciplina de Engenharia de Software Seguro.

TODAS AS 7 ETAPAS ENTREGUES E VALIDADAS

Repositório no GitHub

Acesse o código-fonte, diagramas, suíte de testes e o roteiro do vídeo:

<https://github.com/fadekanaan/es-seguro-gp06>

Engenharia de Software Seguro — UNIPAMPA 2026